

“锁”之四



ReadWriteLock

北京理工大学计算机学院
金旭亮

概述



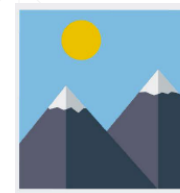
ReentrantReadWriteLock		
m	getOwner()	Thread
m	getQueueLength()	int
m	getQueuedReaderThreads()	Collection<Thread>
m	getQueuedThreads()	Collection<Thread>
m	getQueuedWriterThreads()	Collection<Thread>
m	getReadHoldCount()	int
m	getReadLockCount()	int
m	getWaitQueueLength(Condition)	int
m	getWaitingThreads(Condition)	Collection<Thread>
m	getWriteHoldCount()	int
m	hasQueuedThread(Thread)	boolean
m	hasQueuedThreads()	boolean
m	hasWaiters(Condition)	boolean
m	isFair()	boolean
m	isWriteLocked()	boolean
m	isWriteLockedByCurrentThread()	boolean
m	readLock()	ReadLock
m	toString()	String
m	writeLock()	WriteLock

ReadWriteLock是一个接口，JDK中有一个类ReentrantReadWriteLock实现了此接口。

ReentrantReadWriteLock包容有读锁和写锁两种锁。这两种锁，均实现了Lock接口，所以，都可以使用lock()、unlock()和tryLock()这三种方法。

使用ReadWriteLock对象，可以用于同步多个线程针对特定共享资源的读写操作。

读写锁



读锁是共享的（Shared），一个读线程持有一个读锁的时候并不妨碍其他读线程获得该读锁



写锁是排他的（Exclusive），即一个线程持有写锁的时候其他线程无法获得相应锁的写锁或读锁。



任何一个线程持有一个读锁的时候，其他任何线程都无法获得相应锁的写锁。这就保障了读线程在读取共享变量期间没有其他线程能够对这些变量进行更新，从而使读线程能够读取到相应变量的最新值。

示例分析

定义一个共享资源：

ReadWriteSharedData		
m	ReadWriteSharedData()	
f	data	Integer
f	rwl	ReadWriteLock
f	readData	ThreadLocal<Integer>
m	get()	void
m	put(Integer)	void

data 字段用于存取真正的数据，线程调用 get() 方法读出它的值并输出在控制台窗口中，线程调用 put 方法将新值写到 data 字段中。

此类中的 readData 字段引用一个 ThreadLocal 对象，它用于保存当前线程所读到的数据，这些数据是线程所私有的，互不影响。

```
class ReadWriteSharedData {  
    //共享的数据，只能有一个线程能写该数据，但可以有多线程同时读该数据。  
    private Integer data = null;  
    //用于为每个读线程提供一个本地副本  
    ThreadLocal<Integer> readData=new ThreadLocal<>();  
    //读写锁  
    ReadWriteLock rwl = new ReentrantReadWriteLock();  
    //读操作  
    public void get() {...}  
    //写操作  
    public void put(Integer data) {...}  
}
```

从共享资源中读

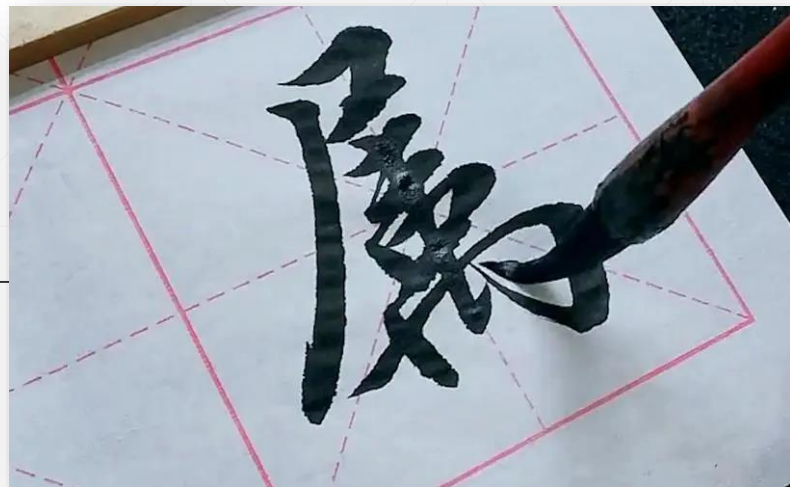


```
//读操作
public void get() {
    //获取读锁
    rwl.readLock().lock();
    try {
        if(!Objects.equals(data, readData.get())){
            System.out.println(Thread.currentThread().getName() + " 读到了新数据 :" + data);
            readData.set(data);
        }
    }
    finally {
        rwl.readLock().unlock(); //释放读锁
    }
}
```

通过对比上次读到的值，
确定是否有新值需要提取

写入共享资源

```
//写操作
public void put(Integer data) {
    //获取写锁
    rwl.writeLock().lock();
    try {
        System.out.println(Thread.currentThread().getName() + " 准备好了写数据!");
        this.data = data;
        System.out.println(Thread.currentThread().getName() + " 写入了数据: " + data);
    } finally {
        rwl.writeLock().unlock(); //释放写锁
    }
}
```



测试代码



```
public static void main(String[] args) {
    final var dataSource = new ReadWriteSharedData();
    //3个读线程
    for (int i = 0; i < 3; i++) {
        new Thread(() -> {
            while(true) {
                dataSource.get();
                try {
                    Thread.sleep((long) (Math.random() * 500));
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }).start();
    }
}
```

```
//1个写线程
new Thread(() -> {
    for (int j = 0; j < 5; j++) {
        try {
            dataSource.put(new Random().nextInt(bound: 10000));
            Thread.sleep((long) (Math.random() * 5000));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    System.out.println("写入结束! ");
    System.exit(status: 0);
}, name: "写入者线程").start();
}
```


运行截图



```
Run: ReadWriteLockTest x
"C:\Program Files\Java\jdk-17\bin\java.exe"
写入者线程 准备好了写数据!
写入者线程 写入了数据: 9508
Thread-1 读到了新数据 :9508
Thread-0 读到了新数据 :9508
Thread-2 读到了新数据 :9508
写入者线程 准备好了写数据!
写入者线程 写入了数据: 2731
Thread-2 读到了新数据 :2731
Thread-0 读到了新数据 :2731
Thread-1 读到了新数据 :2731
```


小结



读写锁适合于在以下条件同时得以满足的场景中使用：



只读操作比写（更新）操作要频繁得多



读线程持有锁的时间比较长